



Akka Distributed Query System

Team: RRR

Rohith Reddy	NU 002641044
Ruby Wang	NU 002610125
Ruchika Shashidhara	NU 002245068

Project Description



Develop a distributed operation that partitions a large dataset and distributes these partitions—with replicas—across worker actors.

Security: Access to projects is controlled by assigning specific roles to users. ACLs define which users or nodes have access to specific resources or perform specific actions.

Dataset: GitHub Events API Payloads (GH Archive)

Processing: Supports broadcasting of small datasets as needed.

Execution: On query submission, the system routes the query to the relevant partitioned actors, processes locally, and aggregates the results.

Use Cases

- Data Analyst inputs query parameters (e.g., date range, event type) and receives a list of matching GitHub event records.
- Data Analyst refines a query by adding filters for specific actors and repositories, and the system updates the result set with aggregated counts.
- Data Analyst selects multiple criteria (such as event type, timestamp, and repository) for a detailed drill-down analysis, and the system displays detailed records for in-depth examination.
- Data Analyst schedules recurring queries for monitoring trends, and the system automatically executes these queries, updating a centralized dashboard.
- Data Analyst compares event trends across several repositories by inputting comparative queries, and the system aggregates and visualizes the data trends for easy analysis.

Methodology

Define data characteristics, performance metrics, and SLA targets.

Requirement Analysis

System Design

Implementation

Testing & Benchmarking

Implement Monitoring & Failover

Create architecture diagrams and design the actor-based model.

Develop Partition Manager, Coordinator, and Worker actors using Akka.

Incorporate replication, distributed query routing, and aggregation.

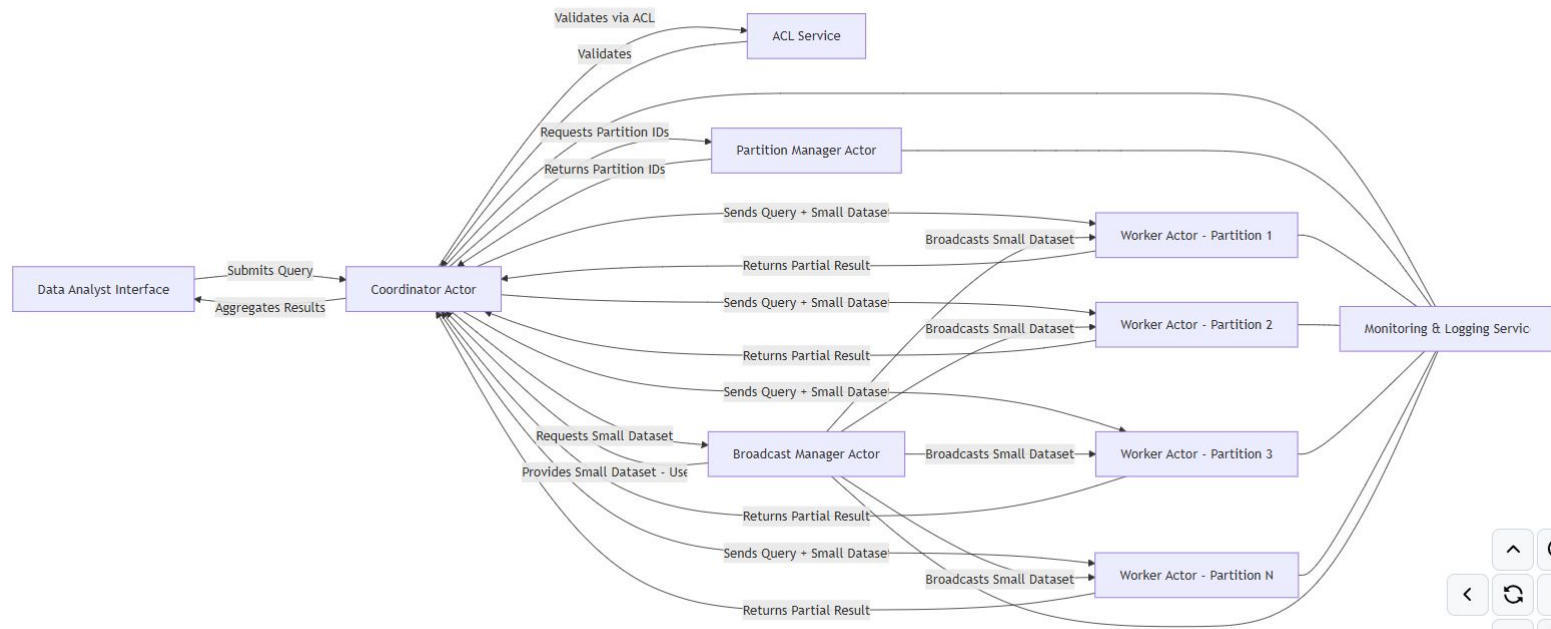
Unit and integration tests using Akka TestKit/Cluster TestKit (Tests were implemented as well as CI)

Implement performance benchmarking using Gatling (Not implemented)

Implement alerts and automatic recovery mechanisms (Not implemented)

Implemented Monitoring using Kamon Metrics (Implemented)

High Level Architecture



Implementation Details



Client: Submits queries to the system and receives results

Coordinator Actor: Manages query routing, aggregates results, and ensures efficient communication

Worker Actors: Process data locally based on the assigned partitions and return partial results

Replication Manager: Maintains replicas of data partitions and handles failover in case of node failure

Broadcast Service: Efficiently distributes small, immutable datasets to all nodes to minimize data redundancy

Data sources

- Dataset: GitHub Events API Payloads - <https://www.gharchive.org/>
- Standardized GitHub event objects with common fields - [GitHub Events](#)
- Includes metadata like event ID, type, actor, repository, payload, and timestamp.
- Captures a variety of GitHub actions (e.g., pushes, pull requests, issues).
- Type: Uncompressed JSON
- Size: 570 MB
- Total Number of Rows: 180387 (Able to process complete dataset)

Sprint Planning



Week 1 (3/24)

- Requirements analysis & project setup
- Architecture design & data model
- Research on Akka & initial prototyping

Week 2 (4/1)

- Develop Partition Manager Actor
- Implement Coordinator Actor
- Create Worker Actor & define message protocols

Week 3 (4/8)

- Integrate dataset partitioning logic
- Develop query routing mechanism
- Implement replication & failover logic

Week 4 (4/15)

- Write unit/integration tests
- Implemented monitoring system
(Reached till here)
- Benchmark performance & optimize
- Implement alerting system

Week 5 (4/22)

- Final integration & code review
- Documentation & sprint wrap-up
- Final testing, deployment simulation

Programming Aspects

- All code will be written in Scala
- Project Repository - <https://github.com/rohithreddykota/force>
- Securing Akka using ACLs and User Management

Acceptance Criteria



- **Average Processing Time:** time < 50 ms; 90th percentile < 100 ms per actor
- Should be able to process ≥ 500 messages/second per actor (eventually)
- **Average Mailbox size:** < 20 messages per actor for 95% of the time
- References: <https://doc.akka.io/libraries/akka-insights/current/instrumentations/akka/akka.html>

Real Time Metrics

Akka Overview

Services > DistribuQuerySystem > Akka Overview

Production



Last 90 minutes



Live

Overview

Operations

JVM

Akka

Force



Throughput

100.4K Messages
18.6/sec Throughput



Akka Status

- Dead Letters 16
- Unhandled Messages 0
- Active Actors 1.2K ± 100%

Relevant Dispatchers

Dispatchers 5

Routers 1

Systems 1

akka.system	name	type	Throughput	Processed Messages	Max Queue Size	Max Active Threads
DistribuQuerySystem	akka.actor.internal-dispatcher	ForkJoinPool	31.8/sec	171.7K	14	8

Challenges

- Massive dataset size exceeds memory capacity for batch jobs
- Long load times and high latency when reading full files
- Difficulty in scaling processing as data grows

Solution: Akka Typed + Streams

- Adopt a streaming pipeline (e.g. Akka Streams) to process events as they arrive
- Balance & back-pressure ensure each worker only handles what it can manage
- Partitioned parallelism distributes work across multiple actors/nodes for horizontal scaling
- On-the-fly filtering & aggregation minimizes I/O and memory footprint

Goals

- ✓ Have a strong understanding of and practical experience with Scala and Akka
- ✓ Efficiently partition & replicate 570 MB data
 - Meet SLA for distributed processing.
- ✓ Deliver a robust, distributed system that processes GitHub events



Thank you